

Anglo-Chinese Junior College
H2 Computing 9569
2022 JC2 Term Assessment Suggested Solutions

1(a)	SQL	NoSQL	
	Has fixed, predefined schema, i.e. fixed data types	No predefined schema, i.e. no fixed data types, thus dynamic and can change easily	
	Data are stored in tables	Data are stored as collections of documents	
	Vertically scalable	Horizontally scalable	
	Stored in central server	Distributed across multiple files/computers	
1(b)	Possible points for SQL: • Since schema is known in advance, SQL is a suitable choice. Flexibility of NoSQL not required. Fixed schema enforces consistency in the tables. • Multiple transactions can occur simultaneously, hence multiple clerks can perform data entry at the same time. • Able to use join to perform complex queries when necessary. Possible points for NoSQL: • Healthcare regulations might change over time. Fixed schema might be too rigid. NoSQL is more flexible and allows for changes. • Queries are expected to be simple, hence database performance is better for NoSQL. • Horizontal scaling allows clinic to improve performance by adding additional servers. That is cheaper than replacing the existing server with a higher performing one as required by SQL. Student should weigh pros and cons and make a final recommendation.		

1(c)	1. Consent Obligation - Only collect, use or disclose personal data for purposes for which an individual has given his or her consent. 2. Purpose Limitation Obligation - An organisation may collect, use or disclose personal data about an individual for the purposes that a reasonable person would consider appropriate in the circumstances and for which the individual has given consent. 3. Notification Obligation - Notify individuals of the purposes for which your organisation is intending to collect, use or disclose their personal data on or before such collection, use or disclosure of personal data. 4. Access and Correction Obligation - Upon request, the personal data of an individual and information about the ways in which his or her personal data has been or may have been used or disclosed within a year before the request should be provided. However, organisations are prohibited from providing an individual access if the provision of the personal data or other information could reasonably be expected to cause harmful effects. Organisations are also required to correct any error or omission in an individual's personal data that is raised by the individual. 5. Accuracy Obligation - Make reasonable effort to ensure that personal data collected by or on behalf of your organisation is accurate and complete, if it is likely to be used to make a decision that affects the individual, or if it is likely to be disclosed to another organisation. 6. Protection Obligation - Make reasonable security arrangements to protect the personal data that your organisation possesses or controls to prevent unauthorised access, collection, use, disclosure or similar risks. 7. Retention Limitation Obligation - Cease retention of personal data or remove the means by which the personal data can be associated with particular individuals when it is no longer necessary for any business or legal purpose.	
--------------------	---	--

	8. Transfer Limitation Obligation - Transfer personal data to another country only according to the requirements prescribed under the regulations, to ensure that the standard of protection provided to the personal data so transferred will be comparable to the protection under the PDPA, unless exempted by the PDPC. 9. Accountability Obligation Make information about your data protection policies, practices and complaints process available on request. Designate a Data Protection Officer to ensure that your organisation complies with the PDPA.	
2(a)	• Produces unique value for every unique input • Time complexity of $O(1)$ • Unable to recover input data from hash value	
2(b)	Chaining: • Every bucket in hash table points to a linked list. • When collision occurs, a new node is created with the new data. The node is then chained to the end of the linked list. • During data retrieval, it is necessary to traverse the linked list to retrieve the desired data. Linear probing: • When collision occurs, the data will be inserted into the next available bucket. • The next available bucket is found by probing the next index number and checking if it's empty. • If the bucket is filled, buckets are sequentially probed until an empty bucket is found. If there are no available buckets, an error is returned. • During data retrieval, the data needs to be searched sequentially starting at the location indicated by the hash value until a bucket with the matching key is found or an empty bucket is accessed.	
2(c)	A: IF <code>MyList[Middle] = SearchValue</code> B: IF <code>MyList[Middle] > SearchValue</code> C: <code>Left ← Middle + 1</code>	

2(d)	Iterative algorithm has constant space requirement regardless of how long the list is. The amount of memory space required for recursive algorithm is depending on the number of recursive calls made. More recursive calls (which usually occurs when list is longer) leads to greater memory use.	
3(a)	Insertion sort: $O(n^2)$ Quick sort: $O(n^2)$ worst case	
3(b)	An algorithm that works in place mutates the input without using any extra memory. As the algorithm executes, the input is usually overwritten by the output, and only a small amount of additional space (e.g. for swaps) is needed for this operation. An algorithm that does not work in place requires more additional memory since new lists are created.	
3(c)	1, 5, 3, 2, 4 1, 3, 5, 2, 4 1, 2, 3, 5, 4 1, 2, 3, 4, 5	
3(d)	The middle item. The list is currently sorted in ascending order. Selecting either the 1st or last item leads to sublists that are always 1 item smaller than the previous sublist, which leads to the worst case scenario for quick sort. Whereas selecting the middle item leads to evenly distributed sublists, which improves on the efficiency.	
3(e)	The above list is almost sorted. For insertion sort, it only needs to make only one swap in one pass to sort the list. Hence the time complexity sort ends up being $O(n)$. For Quicksort, if the code chooses the last (or first element as the pivot), the time complexity ends up being $O(n^2)$ which is slower than insertion sort. Hence the student is correct.	

4(a)	All the data is atomic. For example, member name and member ID are given in separate fields.	
4(b)	There is a transitive dependence since Coach Name depends on Coach ID which in turn depends on Booking ID.	
4(c)	Booking (<u>BookingID</u>, BookingDateTime, BookingVenue, <u>MemberID</u>, <u>CoachID</u>) Coaches (<u>CoachID</u>, CoachName)	
4(d)		
4(e)	SELECT Bookings.BookingDateTime, Bookings.BookingVenue FROM Booking INNER JOIN Members ON Bookings.MemberID = Members.MemberID WHERE Members.MemberName = 'Sally Khoo' ORDER BY Bookings.BookingDateTime ASC Alternative: SELECT BookingDateTime, Bookings.BookingVenue FROM Booking WHERE MemberID IN (SELECT MemberID FROM Members WHERE MemberName = 'Sally Khoo') ORDER BY BookingDateTime ASC	
4(f)	Let the new booking's time be T. Search for bookings with that coach whose time overlaps (e.g. Start time is less than one hour before or after T) regardless of venue If this is found, there is a clash and the coach cannot agree to the new booking. Next, search for bookings with that coach at a different venue whose time is within a certain range of T (e.g. if the coach needs one hour to travel, there must be no bookings that start less than 2 hours before or after T). If this is found, there is a clash and the coach cannot agree to the new booking.	

